COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010 Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	7/21/26	**DATE SUBMITTED:**	7/28/26
NAME:	Vince Andrew Urbano	**INSTRUCTOR:**	Engr. Jimlord Quejado

Seatwork. 3.1 Creating a Simply Linklist

1. PROCEDURE OUTPUT

Screenshot

Code:

HOA3p1 > main.cpp > ...

```cpp
#include <iostream>
#include "simply_LL.h"

int main(){

//creating an empty list
    SimplyList<char>* head = new SimplyList<char>;
    SimplyList<char>* second = new SimplyList<char>;
    SimplyList<char>* third = new SimplyList<char>;
    SimplyList<char>* fourth = new SimplyList<char>;
    SimplyList<char>* fifth = new SimplyList<char>;

    //assign the data
    head->data = 'V';
    head->next = second;

    second->data = 'I';
    second->next = third;

    third->data = 'N';
    third->next = fourth;

    fourth->data = 'C';
    fourth->next = fifth;

    fifth->data = 'E';
    fifth->next = nullptr;

    std::cout<<"Original List"<<std::endl;
    listTraversal(head);

    std::cout<<"Inserting a new node to the head.\n";
    sllinsertHead('Q', &head);
    listTraversal(head);
    std::cout<<"Inserting a new node at the third"<<std::endl;
    sllGeneralInsert('X', head->next->next);
    listTraversal(head);

    std::cout<< "Inserting new node at the end."<<std::endl;
    sllinsertEnd('U', &head);
    listTraversal(head);

    std::cout<<"Deleting a Node"<<std::endl;
    sllDeleteNode('X',&head);
    listTraversal(head);

    std::cout<< "Deleting all the Nodes."<< std::endl;
    sllDeleteList(&head);
```

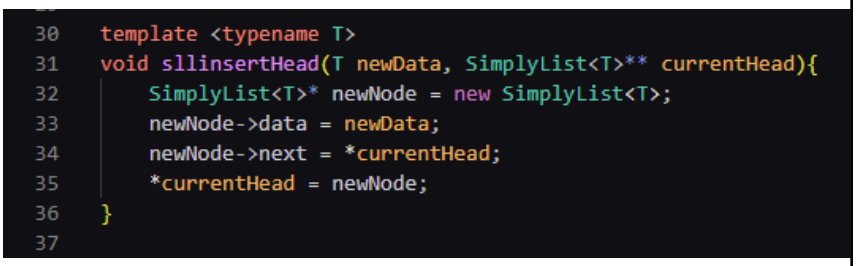
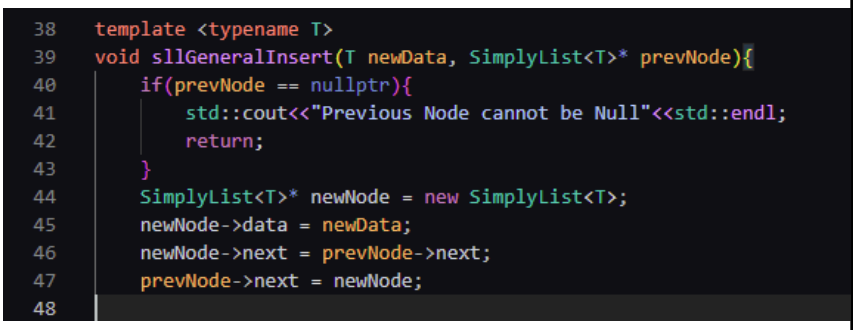
```
49      listTraversal(head);
50
51      return 0;
52  }
```

Output:

```
PS C:\msys64\ucrt64\bin> C:\Users\TIPQC\Documents\CPE_Urbano\HOA3p1\main.exe
Original List
V->I->N->C->E
Inserting a new node to the head.
Q->V->I->N->C->E
Inserting a new node at the third
Q->V->I->X->N->C->E
Inserting new node at the end.
Q->V->I->X->N->C->E->U
Deleting a Node
Q->V->I->N->C->E->U
Deleting all the Nodes.
List is empty
```

Discussion	In this code, we have used different functions from singly_ll.h. For the original list, we have used the listTraversal because the function of it is to list every object. sllInsertHead was used on the second output, we can see that the letter Q was added at the head making it the new head of the linked list. General Insert was used on the third output, generalInsert allows the user to add any object at any position. The fourth output uses the insertEnd function which adds the new object at the end of the list. Deleting nodes allows the user to remove any letters that they want to delete because its function finds the matching letter or characters that was set by the user. Lastly, deleteList is used to delete the whole list.

Table 1-1 Output of Initial/Simple Implementation

Operation	Screenshot
Traversal	
Insertion at head	
Insertion at any part of the list	
Insertion at the end	

```
template <typename T>
void listTraversal(SimplyList<T>*head){

    if(head == nullptr){
        std::cout<<"List is empty"<< std::endl;
        return;
    }
    while(head != nullptr){

        std::cout<<head->data;
        if(head->next != nullptr){
            std::cout<< "->";
        }
        head = head->next;
    }
    std::cout<<std::endl;
}
```

```
template <typename T>
void sllinsertHead(T newData, SimplyList<T>** currentHead){
    SimplyList<T>* newNode = new SimplyList<T>;
    newNode->data = newData;
    newNode->next = *currentHead;
    *currentHead = newNode;
}

```

```
template <typename T>
void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
    if(prevNode == nullptr){
        std::cout<<"Previous Node cannot be Null"<<std::endl;
        return;
    }
    SimplyList<T>* newNode = new SimplyList<T>;
    newNode->data = newData;
    newNode->next = prevNode->next;
    prevNode->next = newNode;

```

```
}

template <typename T>
void sllinsertEnd(T newData, SimplyList<T>** head){

    SimplyList<T>* newNode = new SimplyList<T>;
    SimplyList<T>* currentNode = *head;
    newNode->data = newData;
    newNode->next = nullptr;

    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }

    currentNode->next = newNode;

}

```

Deletion of a node	

```
template <typename T>
void sllDeleteNode(T findData, SimplyList<T>** head){
    if(*head == nullptr) return;

    SimplyList<T>* currNode = *head;
    SimplyList<T>* prevNode = nullptr;

    while(currNode != nullptr  && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

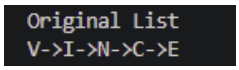
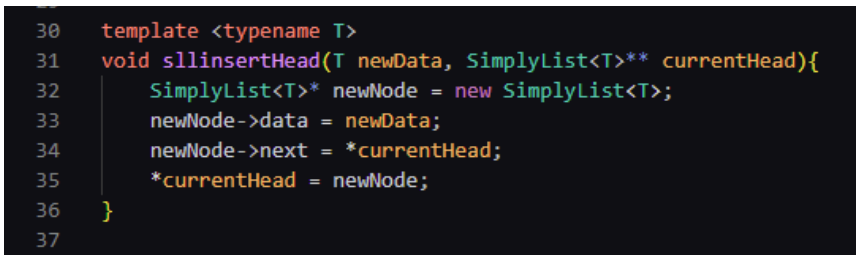
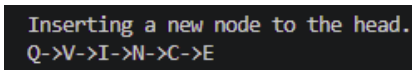
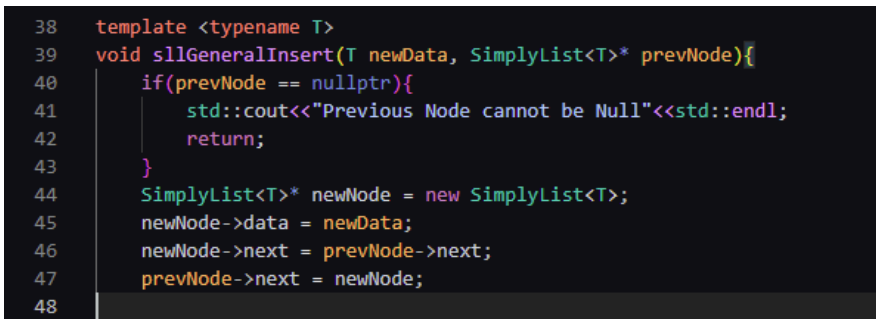
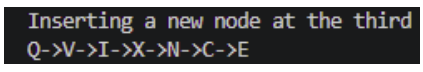
    if (currNode == nullptr) return;

    if (prevNode == nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }

    delete currNode;

}
```

Table 1-2. Code for the List Operations

a	Source Code	
	Console	
b	Source Code	
	Console	
c	Source Code	
	Console	

```
Original List
V->I->N->C->E
```

```
Inserting a new node to the head.
Q->V->I->N->C->E
```

```
Inserting a new node at the third
Q->V->I->X->N->C->E
```

d	Source Code	

```
49  }
50
51  template <typename T>
52  void sllinsertEnd(T newData, SimplyList<T>** head){
53
54      SimplyList<T>* newNode = new SimplyList<T>;
55      SimplyList<T>* currentNode = *head;
56      newNode->data = newData;
57      newNode->next = nullptr;
58
59      while(currentNode->next != nullptr){
60          currentNode = currentNode->next;
61      }
62
63      currentNode->next = newNode;
64
65  }
66
```

	Console	

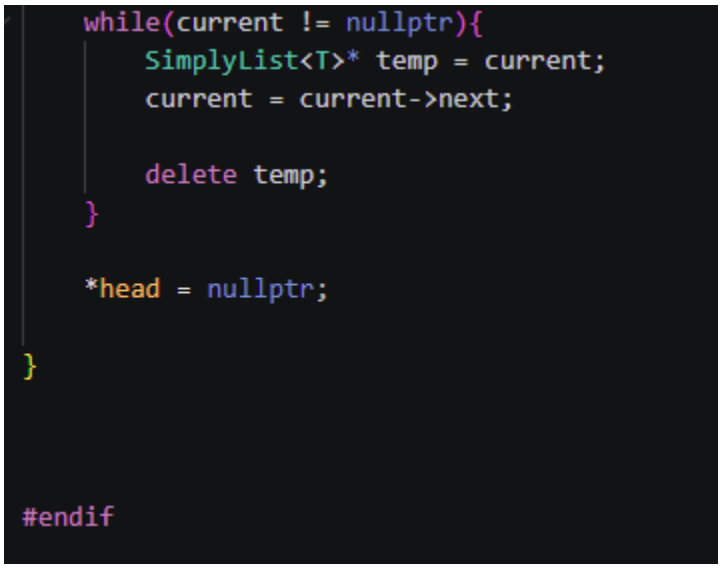
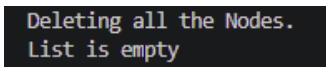
```
Inserting new node at the end.
Q->V->I->X->N->C->E->U
```

e	Source Code	

```
67  template <typename T>
68  void sllDeleteNode(T findData, SimplyList<T>** head){
69      if(*head == nullptr) return;
70
71      SimplyList<T>* currNode = *head;
72      SimplyList<T>* prevNode = nullptr;
73
74      while(currNode != nullptr  && currNode->data != findData){
75          prevNode = currNode;
76          currNode = currNode->next;
77      }
78
79      if (currNode == nullptr) return;
80
81      if (prevNode == nullptr){
82          *head = currNode->next;
83      }else{
84          prevNode->next = currNode->next;
85      }
86
87      delete currNode;
88
89  }
90
```

	Console	

```
Deleting a Node
Q->V->I->N->C->E->U
```

f	Source Code	
	Console	

```
    while(current != nullptr){
        SimplyList<T>* temp = current;
        current = current->next;

        delete temp;
    }

    *head = nullptr;

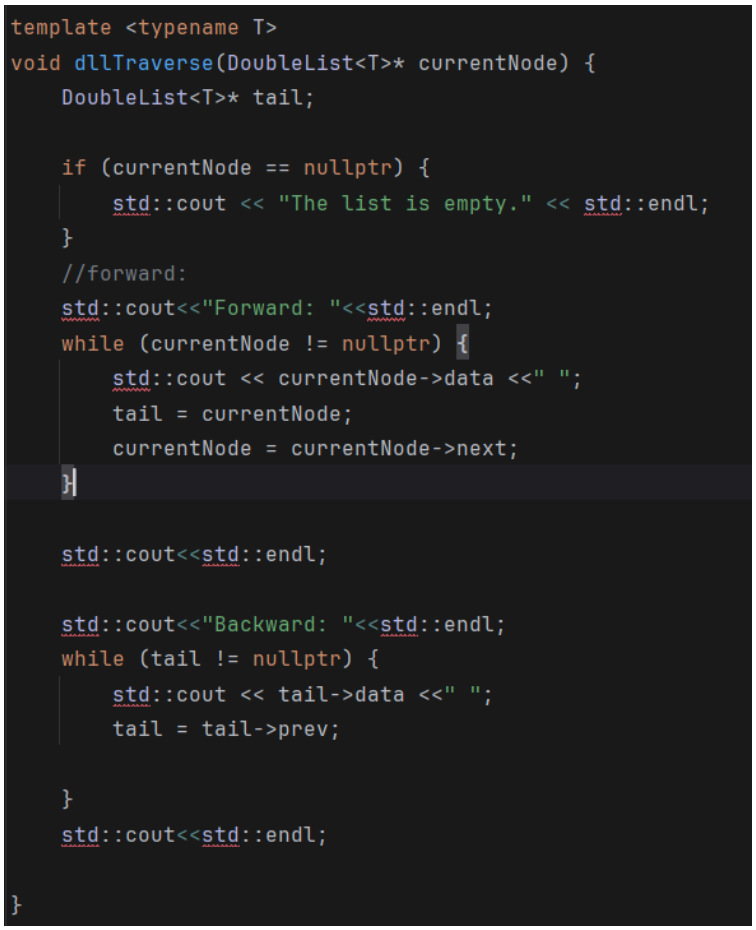
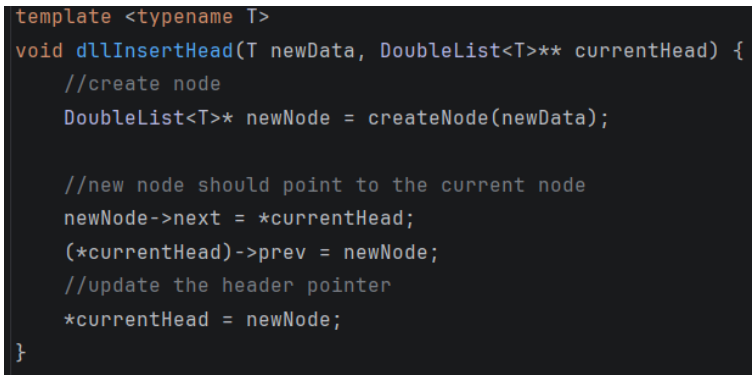
}



#endif
```

```
Deleting all the Nodes.
List is empty
```

Table 1-3 Code and Analysis for Singly Linked Lists

Screenshots	Analysis
	dllTraverse is responsible for printing out the list. First it checks if the list is empty and if it is not the list will print forward and backwards.
	dllInsertHead is used to add a new data at the head of the list.

``
template <typename T>
void dllInsertEnd(T newData, DoubleList<T>* currentHead) {
    DoubleList<T>* newNode = createNode(newData);
 	
    while (currentHead->next != nullptr) {
        currentHead = currentHead->next;
    }
 	
    currentHead->next = newNode;
    newNode->prev = currentHead;
 	
}
``
	dllInsertEnd is responsible for adding new data at the end of the list.

``
template <typename T>
void dllGeneralInsert(T newData, DoubleList<T>* prevNode){
    if (prevNode == nullptr)return;
 	
    DoubleList<T>* newNode = createNode(newData);
    newNode->next = prevNode->next;
    newNode->prev = prevNode;
 	
    if(prevNode->next != nullptr){
        prevNode->next->prev = newNode;
    }
    prevNode->next = newNode;
}
``
	dllGeneralInsert allows the user to add new data on any part of the lists.

```cpp
template <typename T>
void dllDeleteNode(T findData, DoubleList<T>** head) {
    if(*head == nullptr) return;

    DoubleList<T>* currentNode = *head;

    while(currentNode != nullptr  && currentNode->data != findData){
        currentNode = currentNode->next;
    }

    if (currentNode == nullptr) return;

    if (currentNode == *head){
        *head = currentNode->next;
        (*head)->prev = nullptr;
    }else{
        currentNode->prev->next = currentNode->next;
        if (currentNode->next != nullptr) {
            currentNode->next->prev = currentNode->prev;
        }
    }

    delete currentNode;

}
```

dllDeleteNode is responsible for deleting a data that the user wants to be deleted.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Playlist.h

Playlist.h | SUpplementary.cpp | CMakeLists.txt

```cpp
#ifndef Playlist_H
#define Playlist_H

template <typename T>
class playList {
public:
    T data;
    playList<T>* next = nullptr;
};

template <typename T>
void songPlaylist(playList<T>* head) {
    if (head == nullptr) {
        std::cout << "List is empty" << std::endl;
        return;
    }

    playList<T>* current = head;
    do {
        std::cout << current->data;
        if (current->next != head) {
            std::cout << "->";
        }
        current = current->next;
    } while (current != head);

    std::cout << " (back to head: " << head->data << ")" << std::endl;
}

template <typename T>
```

```cpp
template <typename T>
void songPriority(T newData, playList<T>** head) {
    playList<T>* newNode = new playList<T>;
    newNode->data = newData;

    if (*head == nullptr) {
        newNode->next = newNode;
        *head = newNode;
        return;
    }

    playList<T>* last = *head;
    while (last->next != *head) {
        last = last->next;
    }

    newNode->next = *head;
    last->next = newNode;
    *head = newNode;
}

template <typename T>
void addQueue(T newData, playList<T>* prevNode) {
    if (prevNode == nullptr) {
        std::cout << "Previous Node cannot be Null" << std::endl;
        return;
    }

```

```
58      playList<T>* newNode = new playList<T>;
59      newNode->data = newData;
60      newNode->next = prevNode->next;
61      prevNode->next = newNode;
62  }
63
64  template <typename T>
65  void addSong(T newData, playList<T>** head) {
66      if (*head == nullptr) {
67          addSong(newData, head);
68          return;
69      }
70
71      playList<T>* newNode = new playList<T>;
72      newNode->data = newData;
73
74      playList<T>* last = *head;
75      while (last->next != *head) {
76          last = last->next;
77      }
78
79      last->next = newNode;
80      newNode->next = *head;
81  }
82
83  template <typename T>
84  void removeSong(T findData, playList<T>** head) {
```

```
85      if (*head == nullptr) return;
86
87      playList<T>* currNode = *head;
88      playList<T>* prevNode = nullptr;
89
90      if (currNode->data == findData) {
91          if (currNode->next == *head) {
92              delete currNode;
93              *head = nullptr;
94              return;
95          }
96
97          playList<T>* last = *head;
98          while (last->next != *head) {
99              last = last->next;
100         }
101
102         *head = currNode->next;
103         last->next = *head;
104         delete currNode;
105         return;
106     }
107
108     do {
109         prevNode = currNode;
110         currNode = currNode->next;
111     } while (currNode != *head && currNode->data != findData);
```

```
113     if (currNode->data == findData) {
114         prevNode->next = currNode->next;
115         delete currNode;
116     }
117 }
118
119 template <typename T>
120 void deletePlaylist(playList<T>** head) {
121     if (*head == nullptr) return;
122
123     playList<T>* current = *head;
124     playList<T>* nextNode = nullptr;
125
126     playList<T>* last = *head;
127     while (last->next != *head) {
128         last = last->next;
129     }
130     last->next = nullptr;
131
132     while (current != nullptr) {
133         nextNode = current->next;
134         delete current;
135         current = nextNode;
136     }
137
138     *head = nullptr;
```

```
138     *head = nullptr;
139 }
140
141 #endif #ifndef Playlist_H
```

Main:

```cpp
#include <iostream>
#include <string>
#include "Playlist.h"

int main() {



    playList<std::string>* head = new playList<std::string>;
    playList<std::string>* second = new playList<std::string>;
    playList<std::string>* third = new playList<std::string>;
    playList<std::string>* fourth = new playList<std::string>;
    playList<std::string>* fifth = new playList<std::string>;


    head->data = "Ulan";
    head->next = second;

    second->data = "Halik";
    second->next = third;

    third->data = "Alumni Homecoming";
    third->next = fourth;

    fourth->data = "Halaga";
    fourth->next = fifth;

    fifth->data = "Para Sayo";
    fifth->next = head;

```

```
30      std::cout << "Original Playlist" << std::endl;
31      songPlaylist(head);
32
33      std::cout << "Adding Priority Song:" << std::endl;
34      songPriority( newData: std::string( s: "Bulong"), &head);
35      songPlaylist(head);
36
37      std::cout << "\nAdding Song to Queue after Head:" << std::endl;
38      addQueue( newData: std::string( s: "Harana"), head);
39      songPlaylist(head);
40
41      std::cout << "\nAdding Song to End:" << std::endl;
42      addSong( newData: std::string( s: "With A Smile"), &head);
43      songPlaylist(head);
44
45      std::cout << "\nRemoving Song ('Halaga'):" << std::endl;
46      removeSong( findData: std::string( s: "Halaga"), &head);
47      songPlaylist(head);
48
49      std::cout << "\nDeleting Playlist" << std::endl;
50      deletePlaylist(&head);
51      songPlaylist(head);
52
53      return 0;
54  }
```

Output:

```
"C:\Users\Vince Urbano\Documents\Supplementary\HOA3-Supplementary(Playlist)\cmake-build-debug\HOA3_Supp
Original Playlist
Ulan->Halik->Alumni Homecoming->Halaga->Para Sayo (back to head: Ulan)
Adding Priority Song:
Bulong->Ulan->Halik->Alumni Homecoming->Halaga->Para Sayo (back to head: Bulong)

Adding Song to Queue after Head:
Bulong->Harana->Ulan->Halik->Alumni Homecoming->Halaga->Para Sayo (back to head: Bulong)

Adding Song to End:
Bulong->Harana->Ulan->Halik->Alumni Homecoming->Halaga->Para Sayo->With A Smile (back to head: Bulong)

Removing Song ('Halaga'):
Bulong->Harana->Ulan->Halik->Alumni Homecoming->Para Sayo->With A Smile (back to head: Bulong)

Deleting Playlist
List is empty

Process finished with exit code 0
```

Conclusion:

Inside our Playlist.h, we have created our main class named playList and we can see that its data is public meaning it can be accessed by the child class. We have created our first function named songPlaylist, this function is responsible for displaying the playlist that has been made by the user. We can also see in its output the next song that will play and how it is looped back to the head.The function songPriority is responsible for adding a new song at the head and addQueue is used to add a song anywhere on the playlist. addSong is responsible for adding the song at the end of the playlist. removeSong and deletePlaylist is responsible for removing a certain song and deleting the whole playlist in an instant.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Idea generation and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used gemini to clarify parts that I was having a hard time understanding.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.